

# ARQUITETURA DE SOFTWARE – QUESTÕES



## DIRETO DO CONCURSO

### CLIENTE-SERVIDOR

**01.** (UNESC/FESPORTE/ANALISTA DE INFORMÁTICA/2025) Em um sistema de compras online que utiliza a arquitetura cliente/servidor, qual a responsabilidade principal do servidor?

- a) Executar o navegador web e enviar as requisições HTTP para o servidor.
- b) Gerenciar a conexão de rede e garantir a segurança da comunicação entre o cliente e o servidor.
- c) Armazenar os dados dos produtos, processar os pedidos e gerenciar as transações.
- d) Armazenar o histórico de compras do cliente e exibir as recomendações personalizadas.
- e) Exibir a interface gráfica da loja virtual e processar as interações do usuário.



O servidor tem a função principal de armazenar os dados dos produtos, processar pedidos e gerenciar transações. O cliente fica responsável por exibir a interface (navegador) e enviar requisições. Gerenciar conexão e segurança é uma tarefa do servidor, mas não a principal.

**02.** (INSTITUTO ACESSO/CÂMARA DE MANAUS - AM/TÉCNICO EM PROGRAMAÇÃO/2024) Acerca da arquitetura cliente-servidor, utilizada em diversos contextos, incluindo servidores web, bancos de dados e aplicações corporativas, assinale a alternativa INCORRETA.

- a) Pode ser implementada em redes locais (LAN) ou redes de longa distância (WAN).
- b) Servidores podem atender a múltiplos clientes simultaneamente.
- c) O cliente é responsável por armazenar os dados críticos do sistema.
- d) O cliente faz solicitações de serviços, e o servidor responde a essas solicitações.
- e) É amplamente utilizado em aplicações web, como navegadores e servidores web.



O cliente não armazena dados críticos – essa é uma função do servidor. Servidores atendem múltiplos clientes, funcionam em redes locais e remotas, além disso, o modelo é amplamente usado na web.



5m

**03.** (FGV/TJ-MT/ANALISTA JUDICIÁRIO - TI/2024) A arquitetura de sistemas é o design estruturado de um sistema de software que descreve como seus componentes principais se relacionam e interagem para formar um sistema coeso e eficiente. Ela orienta a construção,

manutenção e evolução do software, sendo essencial para garantir que o sistema atenda aos requisitos de desempenho, escalabilidade, segurança e usabilidade. Na arquitetura de sistemas cliente-servidor, o cliente:

- a) É responsável por fornecer serviços de autenticação e controle de acesso.
- b) Processa as requisições e responde diretamente às solicitações do usuário.
- c) Armazena e gerencia centralmente os dados do sistema.
- d) Realiza requisições ao servidor, enviando dados e solicitando informações ou serviços.
- e) Hospeda os serviços principais do sistema, como banco de dados.



O cliente apenas faz requisições e mostra resultados ao usuário. Funções como autenticação, processamento direto sem servidor ou armazenamento central de dados são atribuições do servidor, não do cliente.

## MULTICAMADAS

**04.** (CESPE/CAU-BR/ANALISTA DE INFRAESTRUTURA/2024) Acerca da arquitetura de sistemas de N camadas e das APIs, julgue o próximo item. Na arquitetura de sistemas em N camadas, o sistema é dividido em camadas lógicas, cada uma com uma responsabilidade específica, como apresentação, negócio e dados.



Essa arquitetura divide o sistema em camadas lógicas: apresentação (interface), negócio (regras) e dados (banco de dados). Essa estrutura é fundamental para organizar o código e suas responsabilidades.



10m

**05.** (FUNDATEC/PREFEITURA DE LONDRINA - PR/ANALISTA DE SISTEMAS/2024) Sobre arquitetura de software, assinale a alternativa que corresponde às camadas definidas na arquitetura de três camadas.

- a) Camada de interface, camada de acesso e camada de persistência.
- b) Camada de apresentação, camada de negócios e camada de dados.
- c) Camada de desenho, camada de comunicação e camada de controle.
- d) Camada de cliente, camada de modelo e camada de servidor.
- e) Camada de serviços, camada de aplicativo e camada de banco de dados.



Essa arquitetura é também conhecida como “multicamadas” ou “N camadas”, mas que quando falamos especificamente de três camadas, estamos nos referindo a uma divisão clássica e fundamental.

As três camadas principais são:

1. Camada de Apresentação (ou Interface do Usuário)
  2. Camada de Negócios (ou Lógica de Negócio)
  3. Camada de Dados (ou Acesso a Dados)
- 

**06.** (AVANÇA SP/PREFEITURA DE JUQUITIBA – SP/ANALISTA DE TI/2024) Em todas as redes o objetivo de cada camada é oferecer determinados serviços às camadas superiores, isolando essas camadas dos detalhes de implementação real desses recursos. Quando a camada n de uma máquina se comunica com a camada n de outra máquina, coletivamente, as regras e convenções usadas nesse diálogo são conhecidas como \_\_\_\_\_ da camada n. Analise e indique a alternativa que melhor preenche a lacuna no texto acima:

- a) Laço
- b) Rede
- c) Interface
- d) Protocolo
- e) Arquitetura



Quando camadas de diferentes máquinas precisam se comunicar, elas seguem regras bem definidas chamadas protocolos. Estes protocolos (como HTTP ou TCP) estabelecem como os dados devem ser formatados, transmitidos e interpretados, garantindo que a comunicação ocorra sem erros. Essa padronização é essencial em sistemas distribuídos, onde diferentes componentes precisam trocar informações de forma confiável.

---

**07.** (INSTITUTO ACCESS/BANESTES/ANALISTA EM TI/2024) A camada responsável por estabelecer, gerenciar e encerrar as conexões entre os aplicativos em diferentes dispositivos, por um certo período de comunicação e garantindo que as comunicações possam ser estabelecidas, mantidas e finalizadas de forma adequada. É a camada?

- a) Camada de apresentação.
- b) Camada de sessão.
- c) Camada física.

d) Camada de rede.



A camada de sessão tem um papel crucial no gerenciamento de interações temporárias entre aplicativos. Por exemplo, quando um usuário faz login em um site, essa camada cria e mantém uma sessão ativa, controlando quando ela deve ser encerrada (após período de inatividade ou logout). Isso difere da camada de apresentação (que mostra dados) ou da camada física (que lida com conexões brutas), pois foca especificamente no controle do diálogo entre sistemas.



15m

## MVC

**08.** (CESGRANRIO/BANCO DA AMAZÔNIA/TÉCNICO CIENTÍFICO/2022) Um programador de sistemas computacionais vai utilizar o padrão MVC para desenvolver um aplicativo para um banco. O principal processamento da aplicação vai ser realizado quando o usuário clicar um objeto botão. O evento acionado pelo botão fará uso de um intermediador, que vai preparar a informação e executar o processamento. Este intermediador, na arquitetura MVC, deve ser tratado na camada

- a) Controller
- b) Middleware
- c) Model
- d) Restore
- e) View



Podemos descrever o Controller como o “maestro” do padrão MVC. Quando um usuário clica em um botão (View), o Controller recebe essa ação, coordena a requisição com o Model (que acessa dados e aplica regras de negócio) e, finalmente, atualiza a View com a resposta. O Controller nunca deve conter lógica de negócio ou regras de apresentação – sua única função é orquestrar o fluxo entre Model e View.

**09.** (UFG/TJ-GO/ANALISTA DE SISTEMAS/2024) No modelo MVC, a camada Model é responsável por:

- a) Processar as solicitações de dados do usuário.
- b) Apresentar os dados aos usuários.
- c) Manipular os dados, manter a lógica e as regras do sistema.

d) Analisar a conformidade dos dados conforme regras de negócio.



O Model é o núcleo de inteligência do MVC. Ele encapsula não apenas os dados (como informações de produtos em um e-commerce), mas também todas as regras que os governam (cálculos de frete, validações de estoque). Um erro comum é confundir o Model com “apenas o banco de dados” – na verdade, ele representa o domínio completo do sistema, independente de como os dados são armazenados.

---

**10.** (IBFC/DPE-MT/ANALISTA DE SISTEMAS/2022) Leia a frase abaixo referente ao Modelo MVC. “A camada \_\_\_\_\_ é responsável por intermediar as requisições enviadas pelo \_\_\_\_\_ com as respostas fornecidas pelo \_\_\_\_\_”. Assinale a alternativa que preencha correta e respectivamente as lacunas.

- a) Controller / Model / View
- b) View / Model / Controller
- c) Controller / View / Model
- d) Model / Controller / View



A dinâmica do MVC foi comparada a um ciclo bem definido:

- A View captura interações do usuário (como um clique)
- O Controller interpreta essa ação e solicita processamento ao Model
- O Model executa a lógica e retorna os resultados
- O Controller atualiza a View com esses resultados

Esse fluxo unidirecional evita acoplamento e mantém cada componente com responsabilidades claras.

---

## ORIENTADA A OBJETOS

**11.** (QUADRIX/CFO/ANALISTA DE DESENVOLVIMENTO/2025) A construção de um software começa com seu projeto, fase em que são definidas sua arquitetura, suas estruturas (programas e dados) e a escolha da metodologia a ser adotada. Com base nessa informação, julgue o item seguinte.

Em arquiteturas de software orientadas a objetos, a principal função das classes é armazenar dados, e a interação entre objetos não tem impacto significativo no design do sistema



As classes não são meros “recipientes de dados”, como muitos pensam. Elas combinam atributos (estado) e métodos (comportamento) para modelar entidades completas. Por exemplo, uma classe ContaBancaria, que além de armazenar saldo (dado), possui métodos como sacar() ou transferir() (comportamentos). A interação entre objetos dessas classes é que define a arquitetura do sistema.



20m

**12.** CESPE/CEBRASPE/CNPQ/CESPE/CEBRASPE/2024/CNPQ/Analista em Ciência e Tecnologia Pleno I/Especialidade: Desenvolvimento e Arquitetura de Software)

Julgue o próximo item, no que se refere à arquitetura de sistemas.

Na arquitetura orientada a objeto, os dados e as operações são encapsulados para facilitar a manipulação das informações.



Facilita a manutenção, pois esconde detalhes complexos e expõe apenas o necessário. O encapsulamento, um dos pilares fundamentais da Programação Orientada a Objetos (POO), age como uma camada protetora para o código. Ele realiza isso através da organização do código, agrupando dados (atributos) e ações (métodos) dentro de uma classe. Além disso, o encapsulamento controla o acesso aos dados internos de uma classe, prevenindo assim modificações acidentais ou indesejadas.

**13.** (ACAFE/CELESC/ANALISTA DE SISTEMAS/2024) Um desenvolvedor de software está iniciando um novo projeto e precisa decidir qual paradigma de programação utilizar. O desenvolvedor tem conhecimento em programação procedural, mas nunca utilizou Programação Orientada a Objetos. Assinale a alternativa que NÃO apresenta um dos princípios básicos da Programação Orientada a Objetos (POO).

- a) Generalização: Visa encontrar soluções abrangentes para problemas comuns, utilizando abstração e herança para criar classes genéricas e reutilizáveis.
- b) Abstração: Permite ocultar detalhes de implementação e expor apenas as funcionalidades essenciais de um objeto para o usuário.
- c) Encapsulamento: Combina dados (atributos) e métodos (comportamentos) em uma única unidade, protegendo os dados de acessos indevidos.
- d) Herança: Permite que novas classes herdem atributos e métodos de classes existentes, promovendo reuso de código e organização hierárquica.

e) Polimorfismo: Permite que objetos de diferentes classes respondam ao mesmo método de formas distintas, proporcionando flexibilidade e extensibilidade.



A generalização, embora seja um conceito útil em modelagem de sistemas, não é considerada um dos princípios essenciais da POO. Ela está mais relacionada ao processo de criar classes mais abstratas a partir de classes específicas, mas não é um pilar central como os outros quatro mencionados.

---

**14.** (UERJ/TÉCNICO EM TI/2024) A programação orientada a objetos se caracteriza por uma abordagem distinta de pensar, sobre as necessidades dos softwares. Com relação a esse paradigma de programação, é correto afirmar que o(a):

- a) Reuso consiste na repetição de um código ou trecho de código recorrente.
- b) Complexidade de uso é baixa, devido à menor quantidade de conceitos necessários.
- c) Abstração permite modelar necessidades essenciais do objeto representado.
- d) Encapsulamento permite ter conhecimento da lógica interna realizada para gerar o resultado.



O reuso em POO não se trata de repetir código, mas de reutilizar funcionalidades através de herança e composição. A POO não é simples: ela exige o domínio de múltiplos conceitos (como os quatro pilares) e tem uma curva de aprendizagem acentuada. Além disso, a abstração em POO serve para focar no que é relevante, omitindo detalhes desnecessários e o encapsulamento tem como objetivo proteger a lógica interna de um objeto, expondo apenas o necessário (via métodos públicos).

---

## **GABARITO**

**01. c****02. c****03. d****04. C****05. b****06. d****07. b****08. a****09. c****10. c****11. E****12. C****13. a****14. c**

---

Este material foi elaborado pela equipe pedagógica do Gran Concursos, de acordo com a aula preparada e ministrada pela professora Ana Júlia Batista de Souza.

A presente gravação tem como objetivo auxiliar no acompanhamento e na revisão do conteúdo ministrado na videoaula. Não recomendamos a substituição do estudo em vídeo pela leitura exclusiva deste material.

---